

Documentación Técnica

23 de marzo de 2026

Índice

1. Clustering	4
1.1. Objetivo	4
1.2. Datos y preprocesamiento	4
1.3. Ingeniería de <i>features</i> por conductor	5
1.3.1. Conteos, velocidad media y uso de carril	5
1.3.2. Velocidad relativa	5
1.3.3. Headway (brecha temporal)	6
1.3.4. TTC y tasa de conflictos	6
1.3.5. Cambios de carril	6
1.3.6. Ponderación mensual (opcional)	6
1.3.7. Actividad temporal	6
1.4. Preparación de datos para clustering	7
1.5. Métodos de clustering	7
1.5.1. K-means	7
1.5.2. Gaussian Mixture Models (GMM)	7
1.5.3. HDBSCAN	7
1.6. Conductores frecuentes vs. raros (opcional)	8
1.7. Salidas y artefactos	8
1.8. Variables de clusters agregadas por pórtico e intervalo	8
2. Predicción de accidentes	8
2.1. Objetivo	8
2.2. Fuentes de datos	9
2.3. Discretización temporal y notación	9
2.4. Ingeniería de variables (features)	9
2.4.1. Variables macro de flujo	9
2.4.2. Variables de clusters agregadas (opcional)	9
2.5. Definición del target (etiqueta)	10
2.6. Construcción del dataset	10
2.7. Partición temporal (train/validación/test)	10
2.8. Manejo de desbalance	10
2.9. Modelos	10
2.10. Calibración de umbral con FAR	11
2.11. Métricas de evaluación	11
2.12. Selección de variables y optimización	11
2.13. Experimentos: efecto de variables de cluster	11

2.14. Artefactos	11
3. Drift detection	12
3.1. Objetivo, alcance y relación con el paper	12
3.2. Dependencias y fuentes de datos	12
3.3. Mapa de la interfaz tab a tab	12
3.4. Feature engineering y construcción del target	13
3.5. Preparación de datos: Stage 1 y Stage 2	17
3.6. Selección de variables	17
3.7. Diseño experimental y estrategias	17
3.8. Métricas, tablas y artefactos finales	23
3.9. Validación mediante pruebas	23
4. Introducción	24
5. Preprocesamiento de Datos	24
6. Cálculo de Variables (Nivel Evento)	24
6.1. Headway (h_i)	24
6.2. Velocidad Relativa Indexada ($v_{rel,i}$)	25
6.3. Conflicto y TTC (Time To Collision)	25
7. Agregación por Patante	25
7.1. Variables Agregadas	25
7.1.1. Velocidad Promedio ($\bar{v}_P$)	25
7.1.2. Velocidad Relativa Promedio ($\bar{v}_{rel,P}$)	25
7.1.3. Headway Promedio ($\bar{h}_P$)	25
7.1.4. Tasa de Conflicto (CR_P)	26
7.1.5. Uso de Carriles ($Lane_k$)	26
7.1.6. Tasa de Cambio de Pista (LCR_P)	26
7.2. Estrategias de Consolidación	26
7.2.1. 1. Agregación Global (Default)	26
7.2.2. 2. Ponderación Mensual	26
8. Definición de Conductores Frecuentes	26
9. Implementación en Python	27
9.1. Limpieza de Datos: <code>clean_flujos_for_clustering</code>	27
9.2. Cálculo de Features: <code>Clusterization</code>	27
9.2.1. 1. Métricas Simples y Pre-cálculo	27
9.2.2. 2. Velocidad Relativa	27
9.2.3. 3. Métricas de Interacción (Headway, Conflicto)	28
9.2.4. 4. Cambios de Pista (<code>Lane Changes</code>)	28
9.2.5. 5. Ponderación (Si aplica)	28
10. Alcance y metodología	28
11. Mapa general del pipeline	29

12. Pipeline <i>tab a tab</i> (Streamlit Graph Builder)	29
12.1. Eventos	29
12.2. Feature Engineering	29
12.3. Feature Explorer	30
12.4. Feature Selection	30
12.5. Graph	30
12.6. Visual Graph	30
12.7. Network	30
12.8. Optuna	30
12.9. Balance	30
12.10. Training	31
12.11. Evaluación	31
12.12. History	31
12.13. Experiments	31
13. Modelo de datos del grafo	31
13.1. Aristas y atributos	31
13.2. Etiquetado temporal	31
14. Arquitectura del modelo GNN	32
14.1. Temporal head (opcional)	32
14.2. Registro de variantes GNN (encoder + temporal + aristas)	32
14.2.1. Idea general de comparabilidad	32
14.2.2. Variantes implementadas	33
14.2.3. Codificación de aristas (<code>edge_attr</code>)	33
14.2.4. Heads temporales y ensamblado de secuencias	33
14.2.5. Estrategias de cache temporal (punto crítico)	33
14.2.6. Matriz experimental recomendada	34
14.2.7. Métricas para eventos raros	34
14.2.8. Trazabilidad de variantes	34
15. Entrenamiento y optimización	34
15.1. Flujo del entrenamiento	34
15.2. Train-minibatch y regularización	35
16. GraphSMOTE: síntesis de nodos y aristas	35
16.1. Paso 1: embeddings	35
16.2. Paso 2: SMOTE en embeddings	35
16.3. Paso 3: decodificadores $z \rightarrow x$	35
16.4. Paso 4: inserción de nodos	36
16.5. Paso 5: generación de aristas sintéticas	36
16.6. Modos de operación	36
17. ImGAGN: generación adversarial	36
17.1. Paso 1: homogenización	36
17.2. Paso 2: definir cantidad de sintéticos	36
17.3. Paso 3: generador	37
17.4. Paso 4: discriminador	37
17.5. Paso 5: entrenamiento adversarial	37

17.6. Paso 6: grafo final	37
18. Optuna (búsqueda de hiperparámetros)	37
19. Evaluación, calibración y artefactos	38
20. Auditoría técnica y mejores prácticas	38
20.1. Fortalezas observadas	38
20.2. Riesgos o puntos críticos (actualizado)	38
20.3. Recomendaciones priorizadas (estado actual)	39
21. Anexo: referencias a funciones y archivos	39
22. Implementación de MA-AIRL	39
22.1. Objetivo y alcance	39
22.2. Arquitectura general	39
22.3. Datos expertos y construcción del estado	40
22.4. Acciones expertas (proxy de SUMO)	40
22.5. Modelos de MA-AIRL	40
22.6. Función discriminadora y objetivo	40
22.7. Proxy temporal	41
22.8. Métricas de entrenamiento	41
22.9. Exportación de parámetros a SUMO	41
22.10 Integración en la UI	41
22.11 Archivos modificados	41
22.12 Limitaciones actuales y próximos pasos	41

1. Clustering

1.1. Objetivo

El objetivo del módulo de *clustering* es agrupar conductores (identificados por su matrícula) según indicadores de comportamiento observados en los registros de detección (*flujos*). Estos grupos se usan tanto para análisis exploratorio como para construir variables agregadas por pórtico/intervalo que alimentan modelos posteriores.

1.2. Datos y preprocesamiento

Partimos de un conjunto de observaciones $\{r_j\}_{j=1}^N$, donde cada registro contiene, al menos:

$$r_j = (t_j, p_j, \ell_j, v_j, \text{plate}_j),$$

con t tiempo, p pórtico, ℓ carril, v velocidad y plate la matrícula.

Normalización de matrícula Se normaliza como texto en mayúsculas y se eliminan valores inválidos (`,`, `nan`, `null`, `none`). En la implementación se filtran matrículas con largo fuera de $[5, 6]$.

Limpieza de carril y duplicados El carril se convierte a numérico y se filtra a $\ell \in \{1, 2, 3\}$. Además, se eliminan duplicados exactos usando la llave:

$$(\text{plate}, p, \ell, t).$$

Tratamiento de atípicos en velocidad Para cada grupo (p, ℓ) se calculan cuantiles q_α y q_β sobre la velocidad. Dependiendo del modo:

- **Winsorización:** $v \leftarrow \min(\max(v, q_\alpha), q_\beta)$.
- **Filtrado:** se descartan observaciones con $v \notin [q_\alpha, q_\beta]$.
- **Sin acción:** no se modifica v .

1.3. Ingeniería de *features* por conductor

Para cada conductor i (matrícula), se construye un vector de variables $x_i \in \mathbb{R}^d$ a partir de agregaciones sobre sus pasadas. En la implementación, estas variables se guardan en `Resultados/cluster_features*.duckdb` (tabla `cluster_features`).

1.3.1. Conteos, velocidad media y uso de carril

Sea N_i el número de pasadas del conductor i :

$$N_i = \sum_{j=1}^N \mathbf{1}[\text{plate}_j = i].$$

La velocidad media se define como:

$$\bar{v}_i = \frac{1}{N_i} \sum_{j: \text{plate}_j = i} v_j.$$

Para el uso de carril, definimos proporciones:

$$\pi_{i,k} = \frac{1}{N_i} \sum_{j: \text{plate}_j = i} \mathbf{1}[\ell_j = k], \quad k \in \{1, 2, 3\}.$$

1.3.2. Velocidad relativa

Se define una velocidad media de referencia por intervalo de 5 minutos y p rtico:

$$\bar{v}_{p,\tau} = \frac{1}{|\mathcal{I}_{p,\tau}|} \sum_{j \in \mathcal{I}_{p,\tau}} v_j, \quad \mathcal{I}_{p,\tau} = \{j : p_j = p, \lfloor t_j \rfloor_{5\text{min}} = \tau\}.$$

Para cada pasada, la velocidad relativa es:

$$\rho_j = \frac{v_j}{\bar{v}_{p_j, \lfloor t_j \rfloor_{5\text{min}}}},$$

y por conductor:

$$\bar{\rho}_i = \frac{1}{|\mathcal{J}_i|} \sum_{j \in \mathcal{J}_i} \rho_j, \quad \mathcal{J}_i = \{j : \text{plate}_j = i\}.$$

1.3.3. Headway (brecha temporal)

Para cada grupo (p, ℓ) (y opcionalmente por mes), se ordenan registros por (t, plate) . El *headway* de la observación j se calcula como:

$$h_j = t_j - t_{j-1},$$

en segundos, donde $j - 1$ es el registro anterior dentro del mismo grupo. Headways mayores que un umbral $h_{\text{máx}}$ (por defecto 60s) se tratan como faltantes. El *headway* medio por conductor:

$$\bar{h}_i = \frac{1}{|\mathcal{H}_i|} \sum_{j \in \mathcal{H}_i} h_j, \quad \mathcal{H}_i = \{j : \text{plate}_j = i, h_j > 0, h_j \leq h_{\text{máx}}\}.$$

1.3.4. TTC y tasa de conflictos

Sea $\Delta v_j = v_j - v_{j-1}$ la diferencia de velocidades entre el vehículo j y el anterior en el mismo (p, ℓ) . Para cierres ($\Delta v_j > 0$) se define el *Time-To-Collision*:

$$\text{TTC}_j = \frac{h_j v_j}{\Delta v_j}.$$

Para cada pórtico p se define un umbral τ_p (en el código: `TTC_MAX_BY_PORTICO`). Se contabiliza un conflicto si $\text{TTC}_j < \tau_{p_j}$, y la tasa por conductor:

$$\text{conflict_rate}_i = \frac{\sum_{j \in \mathcal{C}_i} \mathbf{1}[\text{TTC}_j < \tau_{p_j}]}{|\mathcal{C}_i|}, \quad \mathcal{C}_i = \{j : \text{plate}_j = i, h_j > 0\}.$$

En la implementación, para evitar valores extremos, se acota TTC_j por arriba a τ_{p_j} .

1.3.5. Cambios de carril

Ordenando las pasadas del conductor i por tiempo (y opcionalmente por mes), se define:

$$\text{lane_changes}_i = \sum_{m=2}^{N_i} \mathbf{1}[\ell_{i,m} \neq \ell_{i,m-1}], \quad \text{lane_change_rate}_i = \frac{\text{lane_changes}_i}{\text{máx}(N_i - 1, 1)}.$$

1.3.6. Ponderación mensual (opcional)

Cuando se activa ponderación mensual, se calculan variables por mes y luego se consolidan por matrícula mediante promedio ponderado por $N_{i,m}$ (pasadas del conductor i en el mes m):

$$\bar{x}_i = \frac{\sum_m N_{i,m} x_{i,m}}{\sum_m N_{i,m}}.$$

1.3.7. Actividad temporal

Para caracterizar cuán “persistente” es un conductor en el tiempo, se calculan métricas de actividad:

$$n_i^{(\text{days})} = |\{[t_j]_{\text{día}} : \text{plate}_j = i\}|, \quad n_i^{(\text{weeks})} = |\{[t_j]_{\text{semana}} : \text{plate}_j = i\}|, \quad n_i^{(\text{months})} = |\{[t_j]_{\text{mes}} : \text{plate}_j = i\}|.$$

1.4. Preparación de datos para clustering

Se selecciona un subconjunto de columnas numéricas (*features*) y se descartan filas con valores faltantes o infinitos en dichas columnas. Antes de clusterizar se estandarizan las variables con *z-score*:

$$z = \frac{x - \mu}{\sigma},$$

usando `sklearn.preprocessing.StandardScaler`.

1.5. Métodos de clustering

1.5.1. K-means

K-means busca particionar los datos en K clusters minimizando la suma de distancias cuadráticas intra-cluster:

$$\min_{\{c_i\}, \{\mu_k\}} \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2.$$

En la implementación se usa `sklearn.cluster.KMeans` y, para evaluación rápida de K , opcionalmente `MiniBatchKMeans`.

Selección de K (métricas) Se calculan métricas para $K \in [K_{\text{mín}}, K_{\text{máx}}]$:

- **Silhouette:** $s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$.
- **Davies–Bouldin:** promedio de la razón entre dispersión intra-cluster y separación inter-cluster.
- **Calinski–Harabasz:** razón entre dispersión inter-cluster e intra-cluster.

1.5.2. Gaussian Mixture Models (GMM)

Un GMM modela la densidad como mezcla:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k),$$

estimada típicamente vía EM. La selección de K se apoya en criterios de información:

$$\text{AIC} = -2 \log L + 2p, \quad \text{BIC} = -2 \log L + p \log n,$$

donde L es la verosimilitud, p el número de parámetros y n el número de muestras.

1.5.3. HDBSCAN

HDBSCAN es un método basado en densidad que construye una jerarquía de clusters y extrae agrupaciones estables. Parámetros típicos:

- **min_cluster_size:** tamaño mínimo de cluster.
- **min_samples:** control de robustez ante ruido.

Las observaciones marcadas como ruido reciben etiqueta -1 .

1.6. Conductores frecuentes vs. raros (opcional)

Para mejorar robustez cuando existen matrículas con pocas pasadas, el flujo de trabajo permite entrenar el modelo en *conductores frecuentes* (por ejemplo, $N_i \geq N_{\min}$) y luego asignar etiquetas a los raros con un umbral de confianza:

- **K-means**: se calcula la distancia mínima al centroide más cercano; si supera un percentil q estimado en el set frecuente, se etiqueta como desconocido (-1).
- **GMM**: se calcula la probabilidad posterior máxima; si es menor que un umbral γ , se etiqueta como desconocido (-1).

Se guarda además un *confidence score* (`confidence_score`) y un indicador `is_rare`.

1.7. Salidas y artefactos

El módulo genera archivos en `Resultados/`:

- Variables por matrícula en DuckDB: `cluster_features*.duckdb`.
- Etiquetas por matrícula: `cluster_kmeans_kK.csv`, `cluster_gmm_kK.csv`, `cluster_hdbscan.csv`.
- Resumen por cluster (*centroides*): `cluster_summary*.csv`.
- Estadísticos descriptivos por cluster: `cluster_descriptive*.csv`.

1.8. Variables de clusters agregadas por pórtico e intervalo

Además del clustering por matrícula, se construyen variables agregadas por pórtico/intervalo a partir de una tabla de etiquetas (`plate` $\rightarrow$ `cluster_label`). Para cada pórtico p , intervalo τ y cluster c :

$$n_{p,\tau,c} = \#\{j : p_j = p, \lfloor t_j \rfloor_{\Delta} = \tau, \text{cluster}(\text{plate}_j) = c\}, \quad \text{share}_{p,\tau,c} = \frac{n_{p,\tau,c}}{\sum_{c'} n_{p,\tau,c'}}.$$

Opcionalmente se agregan:

- **Flow por hora**: $\text{flow}_{p,\tau,c} = n_{p,\tau,c} \cdot \frac{60}{\Delta} \cdot \frac{1}{L}$, donde Δ es el tamaño del intervalo (minutos) y L el número de carriles asumidos.
- **Velocidad media** por cluster: $\bar{v}_{p,\tau,c}$.
- **Densidad**: $\text{density}_{p,\tau,c} = \text{flow}_{p,\tau,c} / \bar{v}_{p,\tau,c}$ (con manejo defensivo cuando $\bar{v} \leq 0$).
- **Variaciones temporales**: Δspeed , $\Delta \text{density}$ vía diferencias entre intervalos consecutivos.
- **Entropía de mezcla de clusters**: $H_{p,\tau} = -\sum_c \text{share}_{p,\tau,c} \log(\text{share}_{p,\tau,c})$.

2. Predicción de accidentes

2.1. Objetivo

El objetivo es construir y evaluar modelos que predigan la ocurrencia de un accidente en un pórtico, utilizando variables agregadas a partir de detecciones vehiculares (**flujos**) y, opcionalmente, variables derivadas de *clusters* de conductores.

2.2. Fuentes de datos

Flujos (detecciones) Cada registro de detección contiene, al menos, un timestamp t , un p rtico p , una categor a vehicular c y una velocidad v . En el sistema, los flujos se almacenan en DuckDB y se consultan por rango temporal.

Eventos (accidentes) Los accidentes provienen de archivos de eventos. Se filtran por tipo *Accidente* y por *V a* (por defecto, v a expresa). Luego se construye una marca temporal `accidente_time` y se mapea la localizaci n (km, eje, calzada) a p rticos cercanos usando `Datos/Porticos.csv`. El p rtico asignado se guarda como `ultimo_portico`.

2.3. Discretizaci n temporal y notaci n

Se discretiza el tiempo en intervalos de tama o Δ minutos. Para una observaci n con timestamp t , definimos el inicio de intervalo como:

$$\tau(t) = \left\lfloor \frac{t}{\Delta} \right\rfloor \Delta.$$

Las variables se agregan por par (p, τ) , donde p es el p rtico.

2.4. Ingenier a de variables (features)

2.4.1. Variables macro de flujo

Para cada p rtico p , intervalo τ y categor a c , definimos:

$$n_{p,\tau,c} = \#\{\text{detecciones en } (p, \tau, c)\}, \quad \bar{v}_{p,\tau,c} = \frac{1}{n_{p,\tau,c}} \sum v.$$

Con L carriles para normalizaci n, se calcula flujo por hora y densidad:

$$q_{p,\tau,c} = \frac{60}{\Delta} \frac{n_{p,\tau,c}}{L}, \quad k_{p,\tau,c} = \frac{q_{p,\tau,c}}{\bar{v}_{p,\tau,c}}.$$

Opcionalmente se agregan variables din micas v a primera diferencia temporal por p rtico:

$$\Delta \bar{v}_{p,\tau,c} = \bar{v}_{p,\tau,c} - \bar{v}_{p,\tau-\Delta,c}, \quad \Delta k_{p,\tau,c} = k_{p,\tau,c} - k_{p,\tau-\Delta,c}.$$

Finalmente se construye una matriz ancha (*wide*) con columnas del estilo `flow_<cat>`, `speed_<cat>`, `density_<cat>`, etc.

2.4.2. Variables de clusters agregadas (opcional)

Si existe un archivo de etiquetas por matr cula (`plate` $\rightarrow$ `cluster_label`), se unen las detecciones con su cluster correspondiente y se agregan por (p, τ, κ) , donde κ es el identificador de cluster:

$$n_{p,\tau,\kappa} = \#\{\text{detecciones en } (p, \tau, \kappa)\}, \quad \text{share}_{p,\tau,\kappa} = \frac{n_{p,\tau,\kappa}}{\sum_{\kappa'} n_{p,\tau,\kappa'}}.$$

Se derivan, opcionalmente, flujo, velocidad media, densidad y diferencias temporales por cluster, y una medida de mezcla:

$$H_{p,\tau} = - \sum_{\kappa} \text{share}_{p,\tau,\kappa} \log(\text{share}_{p,\tau,\kappa}).$$

Estas variables se almacenan como columnas `cluster_share_*`, `cluster_flow_*`, `cluster_speed_*`, etc.

2.5. Definición del target (etiqueta)

Sea un accidente con timestamp t_a y pórtico asignado p_a . El sistema define un *lead time* de un intervalo: el accidente se etiqueta en el intervalo anterior

$$\tau_a = \tau(t_a) - \Delta,$$

y el target se define como:

$$y_{p,\tau} = \begin{cases} 1, & \text{si existe un accidente con } (p_a, \tau_a) = (p, \tau), \\ 0, & \text{en caso contrario.} \end{cases}$$

Con esto, $y_{p,\tau} = 1$ representa “ocurrirá un accidente en el siguiente intervalo”.

2.6. Construcción del dataset

El dataset final contiene, por fila, un par (p, τ) con variables numéricas y **target**. La construcción es:

1. Calcular variables macro de flujo por (p, τ) .
2. (Opcional) Calcular variables de clusters por (p, τ) y unir las por llave (**portico**, **interval_start**).
3. Agregar **target** mediante unión con el conjunto de pares (p_a, τ_a) .

2.7. Partición temporal (train/validación/test)

Para evitar *leakage* temporal, se realiza un *split* por tiempo usando la columna **interval_start**: el conjunto de timestamps se ordena y se asigna el tramo final al test (proporción **test_size**). En entrenamiento estándar, además se separa validación desde el bloque de train/val con **val_size**.

2.8. Manejo de desbalance

La ocurrencia de accidentes es rara ($y = 1$). Se utilizan dos estrategias:

- **Ponderación de clases**: por ejemplo, Random Forest con **class_weight="balanced"**.
- **SMOTE (opcional)**: se genera un dataset balanceado aplicando SMOTE *solo sobre el conjunto de entrenamiento*. Las filas sintéticas se marcan con **synthetic=True** y no se usan para validación de umbral.

2.9. Modelos

Se consideran los siguientes modelos:

- **Random Forest**: ensamble de árboles con ponderación de clases.
- **XGBoost**: gradiente boosting con objetivo binario logístico.
- **SVM**: clasificador con escalamiento (**StandardScaler**) y salida probabilística.

Cada modelo produce un *score* $s(x) \in \mathbb{R}$ (probabilidad o función de decisión).

2.10. Calibración de umbral con FAR

Se define el *false alarm rate* (FAR) y la sensibilidad:

$$\text{FAR} = \frac{FP}{FP + TN}, \quad \text{Sens} = \frac{TP}{TP + FN}.$$

Se selecciona un umbral θ en el set de validación usando la curva ROC. Con un objetivo $\text{FAR} \leq \alpha$, se elige:

$$\theta = \arg \max_{\theta: \text{FAR}(\theta) \leq \alpha} \text{Sens}(\theta),$$

y luego se evalúa en test con $\hat{y} = \mathbf{1}[s(x) \geq \theta]$.

2.11. Métricas de evaluación

En test se reportan, entre otras:

Accuracy, Precision, Recall, $F1$, FAR, Sens, ROC-AUC,

además de la matriz de confusión.

2.12. Selección de variables y optimización

Importancia de variables Se calcula un ranking de importancia con Random Forest (`feature_importances_`) para ordenar variables.

Optuna + SMOTE Se optimizan hiperparámetros del modelo y de SMOTE. Para cada trial:

1. Aplicar SMOTE sobre train.
2. Entrenar el modelo.
3. Calibrar el umbral con FAR en validación.
4. Evaluar el desempeño (p.ej. $F1$) y seleccionar el mejor trial.

2.13. Experimentos: efecto de variables de cluster

Para evidenciar el efecto de incluir clusters, se comparan dos escenarios:

- **Base:** top- K variables del ranking base (sin clusters).
- **Base+Cluster:** top- K variables del ranking combinado (con clusters).

Se repite para $K = 5, 10, \dots$ hasta el máximo disponible, registrando métricas por configuración.

2.14. Artefactos

El flujo genera artefactos en `Resultados/` (nombres con timestamp/sufijo):

- Features de flujo: `accident_flow_features_*.csv` y/o `.duckdb`.
- Features de clusters: `accident_cluster_features_*.csv`.
- Datasets balanceados: `accident_balanced_*.csv`.
- Resultados Optuna: `optuna_*.json` y `optuna_*_trials.csv`.
- Resultados de experimentos iterativos: `experiments_results_*.csv`.

3. Drift detection

3.1. Objetivo, alcance y relación con el paper

El módulo `src/drift_detection_app.py` implementa un espacio de trabajo de replicación para el paper *Evaluating recalibration strategies for real-time crash prediction: adaptive drift detection vs. cumulative retraining*. El propio archivo declara que concentra la sección completa de *Drift detection* en una sola fuente Python, y organiza la reproducción de la metodología, la construcción de features, la selección de variables, la ejecución experimental y la verificación de cobertura.

El alcance real del módulo va más allá de un detector puntual de drift. Incluye:

- preparación determinista del dataset de crash prediction para el corredor analizado;
- estrategias batch de recalibración anual;
- estrategias adaptativas con ADWIN, ARF y KSWIN;
- persistencia de checkpoints, cachés de tuning y artefactos SMOTE;
- salidas resumidas alineadas con Appendix A.6–A.9 y Figure 7 del paper.

La configuración de referencia del paper se fija con `PAPER_TIME_SPAN = 2018-01-01 to 2024-09-30`. Además, el corredor de foco se restringe al conjunto fijo de pórticos 11, 12, 14, 15 (`DRIFT_FOCUS_PORTICOS`), mientras que el orden de segmentos usado para la ingeniería de variables del artículo es 15→14, 14→12 y 12→11.

3.2. Dependencias y fuentes de datos

El módulo asume como fuente principal de flujos la base `Datos/flujos.duckdb` y la tabla `flujos_duckdb`. Para eventos utiliza archivos CSV bajo `Datos/` procesados con `process_accidentes_df`, y para georreferencia y relación entre pórticos utiliza `Datos/Porticos.csv` a través de `load_porticos`, `find_candidate_porticos` y `get_portico_segments`.

La app también depende de librerías opcionales según la etapa:

- `duckdb` para consulta y persistencia de features;
- `optuna` para tuning de hiperparámetros;
- `imblearn` para SMOTE;
- `river` para ADWIN, KSWIN y `AdaptiveRandomForestClassifier`.

3.3. Mapa de la interfaz tab a tab

La función `main()` crea seis tabs con etiquetas reales `Blueprint`, `Eventos`, `Feature engineering`, `Feature Selection`, `Experiments` y `Coverage`. Cada una corresponde a un bloque funcional explícito:

Blueprint `_render_blueprint_tab()` presenta la matriz de replicación del paper, la tabla de trabajo relacionado (`build_related_work_table()`), las secciones/figuras/tablas cubiertas y el plan de migración desde R a Python (`build_python_migration_review_plan()`).

Eventos `_render_events_tab()` carga uno o múltiples archivos de accidentes desde `Datos/`, procesa la asignación a pórticos con `process_accidentes_df()`, conserva los eventos excluidos por falta de mapeo y construye un resumen por tramo para comparar con *Crash prediction*.

Feature engineering `_render_feature_engineering_tab()` ofrece tres fuentes: cargar DuckDB existente, recalcular features o reutilizar el dataset en memoria. En modo de cálculo, consulta `flujos.duckdb`, genera variables para pórticos, segmentos y carriles, construye `target`, aplica Stage 1 y Stage 2 y exporta un DuckDB con tablas `raw_features` y `clean_features`.

Feature Selection `_render_feature_selection_tab()` ejecuta filtrado por correlación, ranking con Random Forest y persistencia del resultado de selección dentro del mismo archivo DuckDB de features.

Experiments `_render_experiments_tab()` bloquea el conjunto final de variables, permite seleccionar modelos batch, estrategias, variantes ARF/KSWIN, modos de balanceo y semillas secuenciales, y luego invoca `run_recalibration_experiments()` con soporte de *preview*, *resume* y arranque limpio ignorando checkpoints existentes.

Coverage `_render_coverage_tab()` valida que todas las secciones, figuras y tablas declaradas en `ARTICLE_SECTIONS`, `ARTICLE_FIGURES` y `ARTICLE_TABLES` queden mapeadas a artefactos concretos en Python. Esta verificación se resume con `article_coverage_percentage()`.

3.4. Feature engineering y construcción del target

La ingeniería de variables está concentrada en `_compute_drift_article_features()`, con apoyo de `feature_catalog_table()` y `feature_engineering_reference_table()`. El objetivo es replicar la Table 4 del paper y producir un dataset listo para predicción en grilla de 5 minutos, manteniendo exactamente la semántica operativa del código.

Notación operativa Sea $\Delta = 5$ minutos el ancho del intervalo temporal, y sea

$$b(\tau) = \left\lfloor \frac{\tau}{\Delta} \right\rfloor \Delta$$

la función que asigna un timestamp τ al inicio del intervalo correspondiente. Bajo la configuración artículo por defecto, el corredor usa el orden de pórticos

$$P = (15, 14, 12, 11),$$

las categorías

$$C = \{\text{Light}, \text{Heavy}, \text{Motorcycle}\},$$

y los segmentos consecutivos

$$S = \{(15, 14), (14, 12), (12, 11)\}.$$

Cada registro AVI crudo aporta timestamp τ_r , velocidad puntual v_r , categoría c_r , patente π_r , pórtico p_r y carril ℓ_r . La app mapea `CATEGORIA` a `Light/Heavy/Motorcycle` con `DRIFT_ARTICLE_CATEGORY_MAP`, es decir, $\{1 \mapsto \text{Light}, 2 \mapsto \text{Heavy}, 3 \mapsto \text{Heavy}, 4 \mapsto \text{Motorcycle}\}$.

Espacio de variables generado Con el corredor fijo de cuatro pórticos, tres segmentos, tres categorías y tres carriles, la configuración completa genera 260 predictores numéricos antes de agregar **target**:

- 96 variables por pórtico y tipo de vehículo: 4 métricas base + 4 deltas, para 4 pórticos y 3 categorías;
- 8 variables composicionales por pórtico: **ft_motorcycle_*** y **ft_heavy_***;
- 72 variables por segmento y categoría: 5 métricas base + 3 deltas, para 3 segmentos y 3 categorías;
- 21 variables globales por segmento: 4 métricas base + 3 deltas, para 3 segmentos;
- 63 variables por carril de origen: 4 métricas base + 3 deltas, para 3 segmentos y 3 carriles.

Si el usuario restringe pórticos o categorías desde la UI, este espacio se reduce, pero las definiciones matemáticas se mantienen invariantes.

Variables por pórtico y tipo de vehículo Para cada intervalo t , pórtico $p \in P$ y categoría $c \in C$, se define el conjunto de observaciones

$$G_{t,p,c} = \{r : b(\tau_r) = t, p_r = p, c_r = c\}.$$

Sobre ese conjunto se calculan las variables con prefijos **flow_**, **vel_**, **sd_**, **den_**:

$$\begin{aligned} \text{Flow}_{t,p,c} &= |G_{t,p,c}|, \\ \text{Vel}_{t,p,c} &= \frac{1}{|G_{t,p,c}|} \sum_{r \in G_{t,p,c}} v_r, \\ \text{Sd}_{t,p,c} &= \sqrt{\frac{1}{|G_{t,p,c}| - 1} \sum_{r \in G_{t,p,c}} (v_r - \text{Vel}_{t,p,c})^2}, \\ \text{Den}_{t,p,c} &= \begin{cases} \frac{\text{Flow}_{t,p,c}}{\text{Vel}_{t,p,c}}, & \text{si } \text{Vel}_{t,p,c} > 0, \\ \text{NA}, & \text{en otro caso.} \end{cases} \end{aligned}$$

En la implementación real, Flow se rellena con 0 cuando no hay observaciones, mientras que Vel, Sd y Den permanecen vacías cuando el grupo no existe o no tiene suficiente soporte. Además, Sd usa la desviación estándar muestral de **pandas**; por tanto, si sólo hay un vehículo en el grupo, la columna queda en **NaN**.

Los cambios temporales se construyen con primera diferencia dentro de cada serie temporal:

$$\Delta X_{t,p,c} = X_{t,p,c} - X_{t-\Delta,p,c}, \quad X \in \{\text{Flow}, \text{Vel}, \text{Den}, \text{Sd}\}.$$

Esto produce las columnas **delta_flow_***, **delta_vel_***, **delta_den_*** y **delta_sd_***. Para el primer intervalo donde la variable base está definida, el código fija el delta en 0; si la variable base ya nace vacía, el delta también queda vacío.

Fraciones composicionales por p rtico Sobre el mismo intervalo y p rtico, la app calcula el flujo total

$$\text{Flow}_{t,p}^{\text{tot}} = \text{Flow}_{t,p,\text{Light}} + \text{Flow}_{t,p,\text{Heavy}} + \text{Flow}_{t,p,\text{Motorcycle}}.$$

Con ello deriva las fracciones

$$\text{FtMotorcycle}_{t,p} = \frac{\text{Flow}_{t,p,\text{Motorcycle}}}{\text{Flow}_{t,p}^{\text{tot}}}, \quad \text{FtHeavy}_{t,p} = \frac{\text{Flow}_{t,p,\text{Heavy}}}{\text{Flow}_{t,p}^{\text{tot}}}.$$

Estas son las columnas `ft_motorcycle_*` y `ft_heavy_*`. Si el flujo total del p rtico es cero, ambas fracciones quedan en `NaN` en vez de forzarse a cero, porque el denominador no representa una mezcla observada.

Matching por segmento entre p rticos consecutivos Las variables segmentales no se calculan a partir de velocidades locales del p rtico, sino de viajes emparejados entre dos p rticos consecutivos. Para cada segmento $s = (p^-, p^+) \in S$, la distancia d_s se obtiene con `_lookup_article_segment_distance_km()`, priorizando metadatos viales y usando fallback geom trico si hace falta.

La ventana m xima de matching depende de la distancia:

$$\tau_s = \max\left(30, \frac{60 d_s}{20}\right) \text{ minutos},$$

porque `DRIFT_ARTICLE_MIN_MATCH_WINDOW_MINUTES` = 30 y la velocidad de referencia m nima es 20 km/h. Un par upstream-downstream se acepta si:

- comparte la misma patente π ;
- el paso downstream ocurre despu s del upstream y antes de τ_s ;
- la categor a coincide entre ambos extremos;
- el tiempo de viaje es estrictamente positivo.

La implementaci n usa `merge_asof(..., direction="backward")` desde el p rtico downstream hacia el upstream y luego elimina duplicados por `plate_clean` y `time_start`, conservando el primer downstream compatible de cada observaci n upstream. El intervalo del match se ancla al timestamp upstream, es decir, a $b(\tau_{\text{start}})$.

Para cada match m del segmento s , se define el tiempo de viaje

$$\text{TT}_m = \tau_m^{\text{end}} - \tau_m^{\text{start}},$$

y la velocidad espacial

$$\text{SegVel}_m = \begin{cases} \frac{d_s}{\text{TT}_m/1 \text{ hora}}, & \text{si } d_s > 0, \\ \text{NA}, & \text{si no hay distancia v lida.} \end{cases}$$

En paralelo, se calcula el indicador de cambio de carril

$$\text{LC}_m = \mathbf{1}\{\ell_m^{\text{start}} \neq \ell_m^{\text{end}}\},$$

si ambos carriles existen; en caso contrario, el match aporta `NaN` a esa componente.

Variables por segmento y tipo de vehículo Sea $M_{t,s,c}$ el conjunto de matches del intervalo t , segmento s y categoría c . A partir de ellos se generan columnas como `flow_light_15_14`, `vel_heavy_14_12` o `cl_motorcycle_12_11`:

$$\begin{aligned} \text{Flow}_{t,s,c} &= |M_{t,s,c}|, \\ \text{Vel}_{t,s,c} &= \frac{1}{|M_{t,s,c}|} \sum_{m \in M_{t,s,c}} \text{SegVel}_m, \\ \text{Sd}_{t,s,c} &= \sqrt{\frac{1}{|M_{t,s,c}| - 1} \sum_{m \in M_{t,s,c}} (\text{SegVel}_m - \text{Vel}_{t,s,c})^2}, \\ \text{CL}_{t,s,c} &= \frac{1}{|M_{t,s,c}^{LC}|} \sum_{m \in M_{t,s,c}^{LC}} \text{LC}_m, \\ \text{Den}_{t,s,c} &= \begin{cases} \frac{\text{Flow}_{t,s,c}}{\text{Vel}_{t,s,c}}, & \text{si } \text{Vel}_{t,s,c} > 0, \\ \text{NA}, & \text{en otro caso.} \end{cases} \end{aligned}$$

Aquí $M_{t,s,c}^{LC}$ denota el subconjunto con carriles observados en ambos extremos. Los deltas ΔFlow , ΔVel y ΔDen repiten la misma definición de primera diferencia temporal usada a nivel de p rtico. No existe ΔCL en el c digo actual.

Variables globales por segmento y por carril de origen El bloque global del segmento agrega todos los matches del intervalo sin separar por categor a:

$$M_{t,s} = \bigcup_{c \in C} M_{t,s,c}.$$

Sobre $M_{t,s}$ se generan `flow_15_14`, `vel_15_14`, `sd_15_14`, `den_15_14` y sus deltas temporales. Matem ticamente, las f rmulas son id nticas a las del bloque segmental por categor a, pero reemplazando $M_{t,s,c}$ por $M_{t,s}$.

Para capturar heterogeneidad por carril, el mismo matching se reagrupa por carril de origen:

$$M_{t,s,\ell} = \{m \in M_{t,s} : \ell_m^{\text{start}} = \ell\}, \quad \ell \in \{1, 2, 3\}.$$

Sobre ese subconjunto se calculan $\text{Flow}_{t,s,\ell}$, $\text{Vel}_{t,s,\ell}$, $\text{Sd}_{t,s,\ell}$, $\text{Den}_{t,s,\ell}$ y sus deltas, produciendo columnas como `vel_lane2_15_14` o `delta_den_lane3_12_11`. Este bloque es importante porque el propio c digo justifica su inclusi n por la relevancia emp rica de variables por carril en la r plica de Figure 6.

Reglas de borde y consistencia num rica La implementaci n distingue entre ausencia de tr fico y ausencia de medida:

- los flujos se rellenan con 0 cuando el grupo no tiene observaciones;
- velocidades, desviaciones, densidades y *lane change fractions* permanecen en NaN si no existe informaci n suficiente;
- los deltas se calculan sobre la versi n ya pivotada de cada bloque, por lo que respetan el mismo patr n de faltantes;
- cuando el c lculo se hace por lotes en DuckDB, las features est ticas coinciden con el c lculo monol tico, y los deltas se reinician en los bordes de lote con 0 o NaN; esto est  cubierto por `test_drift_batched_flow_features_duckdb_consistency`.

Construcción del target El target se agrega con `_add_interval_accident_target()` después de filtrar accidentes del corredor con `_filter_accidents_for_allowed_porticos()`. Si t_a es el timestamp de un accidente, su marca binaria se asigna al intervalo

$$t^* = b(t_a) - \Delta.$$

Por tanto,

$$y_t = 1 \iff \exists a \text{ tal que } b(t_a) = t + \Delta.$$

Esta definición implementa un *lead time* de un bin: las features de t intentan anticipar accidentes que ocurren en el siguiente intervalo de 5 minutos. La prueba `test_drift_article_features_cover_table4_and_target` valida explícitamente este corrimiento; por ejemplo, un accidente a las 00:06 activa `target = 1` en la fila de las 00:00, no en la de las 00:05.

3.5. Preparación de datos: Stage 1 y Stage 2

La Sección 4.2 del paper se operacionaliza con `apply_missing_data_policy()`, `identify_long_zero_accident_runs()`, `filter_long_zero_accident_runs()` y `run_configurable_preparation_pipeline()`.

- **Stage 1:** elimina variables con más de 1% de valores faltantes y luego elimina intervalos con cualquier predictor faltante en las variables restantes.
- **Stage 2:** elimina ventanas largas sin accidentes, por defecto de al menos 7 días consecutivos con `target = 0`.

La salida incluye un `DataPreparationSummary` con conteo de filas iniciales/finales, variables removidas, ventanas detectadas y filas descartadas. La prueba `test_configurable_preparation_pipeline_respects_filters` valida que ambos stages respondan correctamente a sus selectores.

3.6. Selección de variables

La etapa de selección usa `compute_abs_correlations()`, `drop_highly_correlated_features()` y `rank_features_random_forest()`. Operativamente se ejecuta en dos pasos:

1. remover variables altamente correlacionadas con un umbral configurable;
2. entrenar un Random Forest para ordenar las variables restantes y quedarse con el top- N efectivo para los experimentos.

La misma pestaña genera la réplica de Figure 5 (densidad de $|\rho|$) y Figure 6 (importancias top), y persiste el payload de selección en el DuckDB de features. La prueba `test_feature_selection_payload_persists` cubre esta persistencia.

3.7. Diseño experimental y estrategias

Las estrategias disponibles están definidas por `EXPERIMENT_STRATEGIES`: `static`, `period_aligned`, `cumulative`, `adaptive_adwin`, `adaptive_arf` y `adaptive_kswin`. Los modelos batch soportados por `MODEL_NAMES` son `XGBoost`, `AdaBoost`, `Random Forest` y `NNet`; las variantes online son `ARFmoderate`, `ARFmaj`, `ARFfast`, `KSWINpaper` y `KSWINseasonal`.

Configuración experimental formal Sea $Y_0 < Y_1 < \dots < Y_J$ la secuencia de años observados en el dataset ordenado por `interval_start`. La app toma como año base Y_0 por defecto, salvo que el usuario fije otro. Para cada intervalo t , el par (x_t, y_t) está formado por las features seleccionadas y el target binario de accidente a un paso. El experimento siempre opera sobre el dataset limpio de Stage 1 y Stage 2.

La orquestación principal en `run_recalibration_experiments()` construye bloques experimentales sobre el producto cartesiano de:

- estrategias;
- modelos batch o variantes online;
- modos de balanceo `none/smote` cuando aplica;
- semillas de repetición secuenciales.

Esto significa que el resultado final no es una sola corrida, sino un conjunto de bloques persistidos y reensamblados al final. Formalmente, si B es el conjunto de bloques y R el conjunto de semillas, la cantidad base de ejecuciones es $|B| \times |R|$, a la que se agregan las tareas globales de tuning canónico.

Entrenamiento batch interno y calibración de threshold Las estrategias batch, `adaptive_adwin` y `adaptive_kswin` reutilizan `train_model_with_internal_validation()`. Dado un split de entrenamiento T , el flujo real del código es:

1. dividir T en un split estratificado interno $(T^{\text{fit}}, T^{\text{val}})$ con proporción de validación $\rho = 0,2$ por defecto;
2. sintonizar hiperparámetros sobre T^{fit} con AUC media de validación cruzada estratificada;
3. calibrar el umbral operativo sobre todo T usando scores *out-of-fold*;
4. reentrenar el modelo final con todos los datos de T , y aplicar SMOTE si el bloque lo requiere.

Sea $\Theta_{m,b}$ el espacio de búsqueda del modelo m y del modo de balanceo b . La selección de hiperparámetros resuelve:

$$\hat{\theta} = \arg \max_{\theta \in \Theta_{m,b}} \frac{1}{K} \sum_{k=1}^K \text{AUC}_k(\theta),$$

donde K es el número efectivo de folds estratificados. Si `balance_mode = smote`, el propio espacio de búsqueda incorpora `sampling_strategy` y `k_neighbors`; si `balance_mode = none`, sólo se buscan hiperparámetros del modelo.

Una vez elegido $\hat{\theta}$, el código genera scores cruzados s_i^{cv} para cada fila $i \in T$ y selecciona el umbral con el criterio de Youden:

$$\hat{\tau} = \arg \max_{\tau} [\text{TPR}(\tau) - \text{FPR}(\tau)] = \arg \max_{\tau} [\text{Sensitivity}(\tau) + \text{Specificity}(\tau) - 1].$$

Las predicciones duras se definen como

$$\hat{y}_i = \mathbf{1}\{s_i \geq \hat{\tau}\}.$$

Si la calibración *out-of-fold* falla por falta de variabilidad, el código cae a un esquema de respaldo usando el holdout interno T^{val} .

Política de estabilidad para NNet El modelo NNet ya no se entrena sobre variables crudas. Después de la imputación por mediana del split de entrenamiento activo, el código ajusta un `StandardScaler` exclusivamente sobre la partición de entrenamiento correspondiente y transforma validación, test o stream con esos mismos parámetros. Para cada predictor x_j , la transformación aplicada es

$$z_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j},$$

donde μ_j y σ_j se estiman sólo con el subconjunto de entrenamiento del fold, del holdout interno o de la ventana de reentrenamiento que esté activa. Esto evita leakage temporal o entre folds, y la misma lógica se reutiliza en evaluación anual, `adaptive_adwin` y `adaptive_kswin`.

En términos de optimización, `MLPClassifier` se ejecuta con `solver = adam`, `early_stopping = True`, `validation_fraction = 0.1`, `n_iter_no_change = 20` y `tol = 1e-4`. Además, la grilla de `maxit` se endurece a `{300,500}` en *fast mode* y a `{300,500,800}` en modo completo. Los `ConvergenceWarning` de `scikit-learn` no se dejan escapar a terminal: se capturan como señal interna del pipeline.

Metodológicamente, un fold de validación cruzada de NNet que no converge se considera inválido y no puede sostener un trial ganador. Si todos los folds relevantes fallan, el artefacto de tuning queda marcado con `has_valid_trial = false`. Durante el refit final, el código intenta una sola vez un reentrenamiento adicional con `max_iter` duplicado, acotado superiormente por 1000. Si ese reintento tampoco converge, el bloque experimental no se aborta: se persiste como `skipped` y el `execution_log` registra eventos explícitos como `nnet_fold_non_converged`, `nnet_trial_invalid`, `nnet_final_refit_retry` o `nnet_final_refit_skipped`. Los trials persistidos también conservan `converged`, `n_non_converged_folds`, `n_valid_folds` y `n_iter_final` para inspección online.

Balanceo con SMOTE El modo `smote` nunca se aplica sobre test o stream, sólo sobre folds de entrenamiento y sobre el refit final. Si la clase minoritaria tiene menos de dos ejemplos, o si la razón minoritaria/mayoritaria ya está por encima del objetivo, el balanceo no se ejecuta. En caso contrario, SMOTE construye un conjunto balanceado $(X^{\text{bal}}, y^{\text{bal}})$ con:

$$\frac{N_{\text{minor}}^{\text{new}}}{N_{\text{major}}} \approx \text{sampling_strategy},$$

y el vecindario efectivo satisface

$$k_{\text{eff}} = \min(k_{\text{neighbors}}, N_{\text{minor}} - 1).$$

Los artefactos SMOTE pueden persistirse y reutilizarse por bloque mediante `_load_or_create_smote_artifact()`.

Estrategias batch anuales `run_yearly_strategy()` implementa tres reglas de entrenamiento para cada año objetivo Y_j , $j \geq 1$:

$$\mathcal{T}_j^{\text{static}} = \{t : \text{year}(t) = Y_0\},$$

$$\mathcal{T}_j^{\text{period}} = \{t : \text{year}(t) = Y_j - 1\},$$

$$\mathcal{T}_j^{\text{cum}} = \{t : \text{year}(t) \leq Y_j - 1\},$$

y el conjunto de evaluación es siempre

$$\mathcal{E}_j = \{t : \text{year}(t) = Y_j\}.$$

Cada fila de salida contiene `training_year`, `prediction_year`, `model`, `balance_mode`, métricas, umbral, tiempo de entrenamiento y tamaños `n_train/n_test`. En la estrategia `static`, como $\mathcal{T}_j^{\text{static}}$ es idéntico para todo j , el código cachea y reutiliza el bundle entrenado para evitar tuning y refit redundantes; esto está cubierto por `test_run_yearly_strategy_reuses_static_training_bundle`.

El resultado esperado de este bloque es:

- una fila en `yearly_results` por combinación (estrategia, modelo, balance, Y_j , seed);
- una curva ROC segmentada por año objetivo en `roc_payload`;
- tablas Appendix A.6, A.7 y A.8 para `static`, `period_aligned` y `cumulative`, respectivamente.

Recalibración adaptativa con ADWIN `run_adaptive_strategy()` entrena primero un modelo batch sobre el año base Y_0 y luego procesa cronológicamente todo el stream posterior. Para cada fila del stream se calcula un score s_t , una predicción dura $\hat{y}_t = \mathbf{1}\{s_t \geq \hat{\tau}\}$ y una señal de error binario

$$e_t = \mathbf{1}\{\hat{y}_t \neq y_t\}.$$

La clase `SimpleADWIN` mantiene una ventana W de errores. Para cada corte candidato $W = W_0 \cup W_1$, con tamaños n_0 y n_1 , define:

$$\mu_0 = \frac{1}{n_0} \sum_{e \in W_0} e, \quad \mu_1 = \frac{1}{n_1} \sum_{e \in W_1} e,$$

$$m = \left(\frac{1}{n_0} + \frac{1}{n_1} \right)^{-1}, \quad \epsilon = \sqrt{\frac{1}{2m} \log \left(\max \left(1, 0000001, \frac{4 \log n}{\delta} \right) \right)},$$

donde $n = |W|$ y $\delta = \text{adwin_delta}$. Se declara drift si

$$|\mu_0 - \mu_1| > \epsilon.$$

Cuando eso ocurre, el código:

- cierra el segmento de predicción acumulado hasta t ;
- registra `drift_date`, `W`, `W0`, `W1` y `remaining_periods`;
- intenta recalibrar el modelo usando las últimas $|W_1|$ observaciones del stream.

El reentrenamiento sólo ocurre si la ventana W_1 tiene al menos $\max(20, \lfloor \text{min_window}/10 \rfloor)$ filas, o el `min_retrain_size` que fije el usuario, y contiene ambas clases. Si no se cumple esa condición, el detector igual corta el segmento, pero el modelo vigente continúa. El resultado esperado es una tabla `adaptive_results` con una fila por drift detectado más un segmento final con `remaining_periods = 0`; estas filas alimentan la parte ADWIN de Appendix A.9.

Adaptive Random Forest `run_arf_strategy()` implementa una estrategia genuinamente on-line. Sobre el año base Y_0 , la función `train_arf_with_internal_validation()` separa un bloque final de validación cronológica y usa el bloque inicial para *warm-up* del ensamble ARF. Luego predice secuencialmente sobre el bloque de validación, aprende inmediatamente cada etiqueta y fija $\hat{\tau}$ con el mismo criterio de Youden.

Durante el stream posterior, el modelo se actualiza observación a observación:

$$s_t = f_{t-1}(x_t), \quad f_t = \mathcal{U}(f_{t-1}, x_t, y_t),$$

donde $\mathcal{U}$ representa el paso online `learn_one`. A diferencia de ADWIN y KSWIN, no hay un reentrenamiento batch externo tras un drift: la adaptación está internalizada en el bosque. Por eso el código no duplica esta estrategia por `balance_mode`; siempre usa `not_applicable`.

Los resultados se segmentan por año de predicción, no por drift externo. Cada fila esperada contiene:

- `prediction_year`;
- `segment_rows`;
- `n_internal_drifts`;
- `n_internal_warnings`;
- métricas y umbral operativo.

En consecuencia, Appendix A.9 muestra a ARF como segmentos anuales online, no como ventanas $W/W_0/W_1$. Las pruebas `test_run_arf_strategy_variants` y `test_recalibration_experiments_support_adapt` verifican esa salida.

Recalibración adaptativa con KSWIN `run_kswin_strategy()` separa claramente el problema de detección de drift covariable del problema de predicción. Primero entrena un modelo batch sobre Y_0 . Luego selecciona las variables monitorizadas con `_select_kswin_monitor_columns()`, que en la implementación actual toma las primeras **top-k** columnas numéricas del orden de `feature_cols`. No es un ranking online: es una selección determinista de las features ya aprobadas por Feature Selection.

Para cada feature monitorizada f , KSWIN puede operar en dos espacios:

- **KSWINpaper**: usa el valor crudo $x_{t,f}$;
- **KSWINseasonal**: usa un z-score estacional

$$z_{t,f} = \frac{x_{t,f} - \mu_{d(t),h(t),f}}{\sigma_{d(t),h(t),f}},$$

con fallback a perfiles por hora y luego a perfiles globales del año base.

Cada detector usa $\alpha = 10^{-4}$, `window_size` = 300 y `stat_size` = 30, por lo que Appendix A.9 reporta

$$W = 300, \quad W_0 = 270, \quad W_1 = 30$$

como tamaños fijos del detector, no como ventanas aprendidas dinámicamente.

Sea F_t el conjunto de features cuyo detector dispara drift en el instante t . La condición de drift del bloque es:

$$|F_t| \geq \text{vote_threshold}.$$

Cuando eso ocurre, se cierra el segmento de predicción, se registran `vote_count`, `detected_features`, `monitored_features`, y se selecciona una ventana de recalibración temporal:

$$\mathcal{R}_t = \{i : t_i \geq t - D\},$$

donde $D = \text{kswin_retrain_days}$. Si $|\mathcal{R}_t| < \text{kswin_min_retrain_rows}$, el código usa las últimas `min_rows` observaciones; si esa ventana tiene una sola clase, se expande a todo lo observado hasta

t. Tras el refit batch, los detectores KSWIN se reconstruyen y se recalientan con la ventana de reentrenamiento.

El resultado esperado de KSWIN es una fila por drift votado más un segmento final, con columnas adicionales como `vote_threshold`, `monitor_feature_count`, `retrain_rows` y `retrain_positive_rows`. Las pruebas `test_run_kswin_strategy_variants`, `test_recalibration_experiments_support_adaptive_kswin` y `test_kswin_optuna_studies_capture_base_and_retrains_with_balance_mode` validan precisamente ese comportamiento.

Semillas, tuning global, persistencia y reanudación `run_recalibration_experiments()` no vuelve a tunear desde cero cada bloque batch. Antes de lanzar los bloques, construye tareas de tuning canónico sobre el split base del año Y_0 para cada combinación (modelo, balance) que vaya a ser usada por estrategias batch, ADWIN o KSWIN. Ese tuning global se persiste en `tuning_dir` y luego se inyecta como `fixed_params/fixed_tuning_artifact` en los bloques posteriores. En términos metodológicos, esto impone comparabilidad entre estrategias que comparten familia de modelo, porque todas parten del mismo $\hat{\theta}$ canónico por seed y modo de balanceo.

Cada bloque persiste:

- filas anuales o adaptativas;
- ROC payload segmentado;
- su propio `execution_log`;
- referencias a tuning y a artefactos SMOTE.

El `manifest.json` central mantiene progreso, bloques completados, cachés disponibles y errores. Si una corrida falla a mitad de camino, la siguiente ejecución puede reanudar desde el checkpoint y reutilizar tuning/SMOTE ya calculados; esto está cubierto por `test_preview_recalibration_checkpoint_detects_re` y `test_recalibration_experiments_resume_from_failed_checkpoint`.

Para NNet, la compatibilidad de caché no depende sólo del dataset y de la grilla. Tanto el `run_id` como el `tuning_key` incorporan una firma de versión de la política de entrenamiento (`NNET_TRAINING_POLICY_VERSION`). Por ello, si cambian el escalado, la estrategia de *early stopping* o las reglas de convergencia, los checkpoints y tuning caches previos dejan de ser compatibles y no se reutilizan de manera silenciosa.

Resultados esperados Metodológicamente, la salida completa del experimento debe contener siete productos coherentes entre sí:

- `yearly_results`: una fila por split anual batch, con `strategy`, `prediction_year`, `model`, `balance_mode`, métricas, umbral y tamaños muestrales;
- `adaptive_results`: una fila por segmento adaptativo, ya sea delimitado por drift ADWIN/KSWIN o por año en ARF;
- `summary`: medias por (`strategy`, `model`, `balance_mode`), incluyendo `n_segments` y `n_repetitions`;
- `appendix_tables`: A.6 para `static`, A.7 para `period_aligned`, A.8 para `cumulative` y A.9 para resultados adaptativos;
- `appendix_tables_mean`: las mismas tablas, pero promediadas por semillas secuenciales y enriquecidas con `seed_list` y `run_orders`;

- **average_roc**: curvas ROC medias tipo Figure 7, construidas interpolando cada curva individual sobre una grilla uniforme de 101 puntos en FPR y promediando la TPR:

$$\overline{\text{TPR}}_g(u) = \frac{1}{J_g} \sum_{j=1}^{J_g} \widetilde{\text{TPR}}_{g,j}(u);$$

- **run_manifest** y **execution_log**: trazabilidad reproducible del experimento, incluyendo configuración, seeds, thresholds, tuning, checkpoints, errores y consumo de recursos.

Las pruebas `test_end_to_end_recalibration_and_average_roc`, `test_recalibration_experiments_compare_balance_modes_end_to_end` y `test_recalibration_experiments_persist_optuna_json` verifican precisamente que ese paquete de resultados exista, sea consistente y pueda persistirse como JSON final.

3.8. Métricas, tablas y artefactos finales

La evaluación usa cuatro métricas base calculadas por `compute_classification_metrics()`: `auc`, `sensitivity`, `specificity` y `error_rate`, además del `threshold` operativo. Sobre esas tablas, el módulo construye:

- resumen consolidado con `summarize_results()`;
- tablas Appendix A.6–A.9 con `format_appendix_tables()`;
- tablas promedio por semillas con `format_appendix_tables_mean()`;
- curva ROC promedio tipo Figure 7 con `build_average_roc_curves()`.

La persistencia principal se divide en tres niveles:

- features exportadas en DuckDB bajo `Resultados/`;
- checkpoints por corrida en `Resultados/drift_recalibration_runs/`, con `manifest.json`, bloques persistidos, tuning cache y artefactos SMOTE;
- JSON final de resultados con `_persist_drift_recalibration_json()`, guardado como `drift_recalibration_optuna.json`.

El `run_manifest` registra configuración, semillas, estrategias, tamaños de grilla, features activas y rutas de checkpoint. El `execution_log` almacena trazabilidad de tuning, entrenamiento, errores, reentrenos adaptativos y consumo de recursos.

3.9. Validación mediante pruebas

El archivo `tests/test_drift_detection_app.py` entrega cobertura funcional amplia del pipeline. Entre los casos más relevantes:

- `test_article_coverage_is_100_percent`;
- `test_drift_article_features_cover_table4_and_target_shift`;
- `test_drift_batched_flow_features_duckdb_consistency`;
- `test_end_to_end_recalibration_and_average_roc`;
- `test_recalibration_experiments_compare_balance_modes_end_to_end`;

- `test_recalibration_experiments_persist_optuna_json;`
- `test_recalibration_experiments_resume_from_failed_checkpoint;`
- `test_recalibration_experiments_support_adaptive_arf;`
- `test_recalibration_experiments_support_adaptive_kswin.`

En conjunto, estas pruebas confirman que la documentación de *Drift detection* no describe un diseño aspiracional, sino el comportamiento realmente implementado en el repositorio.

4. Introducción

Este documento detalla el proceso de ingeniería de características (*feature engineering*) utilizado para agrupar conductores (patentes) en función de su comportamiento en la autopista. El objetivo es transformar los registros brutos de flujos vehiculares en un vector de características único por patente.

5. Preprocesamiento de Datos

Antes de calcular las variables, los datos de flujo (F) pasan por una etapa de limpieza:

1. **Normalización de Patentes:** Se eliminan caracteres no alfanuméricos y se estandariza a mayúsculas. Patentes inválidas (ej. "NULL", "NAN") o con longitud fuera del rango [5, 6] son descartadas.
2. **Filtrado de Carriles:** Se consideran únicamente los carriles 1, 2 y 3.
3. **Eliminación de Duplicados:** Se eliminan registros duplicados considerando la tupla (*Patente, Portico, Carril*).
4. **Tratamiento de Outliers (Velocidad):** Se aplica *Winsorization* a la velocidad por cada grupo de (*Portico, Carril*), limitando los valores extremos a los percentiles 1% y 99%.

6. Cálculo de Variables (Nivel Evento)

Para cada evento i (un paso de un vehículo por un pórtico), se calculan métricas intermedias. Sea v_i la velocidad del evento i en el instante t_i .

6.1. Headway (h_i)

El *headway* se calcula como la diferencia de tiempo con el vehículo precedente ($i-1$) en el **mismo carril y pórtico**. Se requiere que los datos estén ordenados temporalmente.

$$h_i = t_i - t_{i-1} \tag{1}$$

Si $h_i > 60$ segundos, se considera que no hay interacción cercana y el valor se descarta (NaN) para efectos de cálculo de conflicto, aunque puede contabilizarse en promedios si se desea.

6.2. Velocidad Relativa Indexada ($v_{rel,i}$)

Para normalizar el comportamiento según las condiciones del tráfico, se compara la velocidad del vehículo con la velocidad promedio del flujo en ese p rtico durante un intervalo de 5 minutos, denotada como $\bar{V}_{5min}$.

$$v_{rel,i} = \frac{v_i}{\bar{V}_{5min}} \quad (2)$$

- $v_{rel,i} > 1$: Conductor m s r pido que el flujo promedio.
- $v_{rel,i} < 1$: Conductor m s lento que el flujo promedio.

6.3. Conflicto y TTC (Time To Collision)

Se eval a el riesgo de colisi n con el v h culo precedente. El TTC_i se calcula solo si el v h culo i viaja m s r pido que el v h culo anterior ($v_i > v_{i-1}$) y el headway es v lido.

$$TTC_i = \frac{h_i \cdot v_i}{v_i - v_{i-1}} \quad (3)$$

Se define un evento de **conflicto** (c_i) si el TTC_i es menor a un umbral espec fico por p rtico (TTC_{max}), definido emp ricamente.

$$c_i = \begin{cases} 1 & \text{si } TTC_i < TTC_{max} \\ 0 & \text{si } TTC_i \geq TTC_{max} \text{ o } v_i \leq v_{i-1} \end{cases} \quad (4)$$

7. Agregaci n por Patente

El objetivo es obtener un vector  nico por conductor P . El proceso de agregaci n puede realizarse de dos formas: Global o Ponderada Mensual.

Sea N_P el n mero total de pasadas registradas para la patente P .

7.1. Variables Agregadas

7.1.1. Velocidad Promedio ($\bar{v}_P$)

$$\bar{v}_P = \frac{1}{N_P} \sum_{i \in P} v_i \quad (5)$$

7.1.2. Velocidad Relativa Promedio ($\bar{v}_{rel,P}$)

$$\bar{v}_{rel,P} = \frac{1}{N_{rel}} \sum_{i \in P} v_{rel,i} \quad (6)$$

Donde N_{rel} es el n mero de eventos con velocidad relativa v lida.

7.1.3. Headway Promedio ($\bar{h}_P$)

$$\bar{h}_P = \frac{1}{N_h} \sum_{i \in P, h_i \leq 60} h_i \quad (7)$$

7.1.4. Tasa de Conflicto (CR_P)

Proporción de eventos con headway válido que resultaron en conflicto.

$$CR_P = \frac{\sum_{i \in P} c_i}{N_h} \quad (8)$$

7.1.5. Uso de Carriles ($Lane_k$)

Proporción de uso del carril $k \in \{1, 2, 3\}$.

$$Lane_k = \frac{\text{Conteo de pasadas en carril } k}{N_P} \quad (9)$$

7.1.6. Tasa de Cambio de Pista (LCR_P)

Se estima contando cuántas veces el vehículo cambió de carril entre pódicos consecutivos. Sea L_j el carril en el evento j (ordenado por tiempo).

$$Cambios = \sum_{j=2}^{N_P} \mathbb{I}(L_j \neq L_{j-1}) \quad (10)$$

$$LCR_P = \frac{Cambios}{N_P - 1} \quad (11)$$

Nota: Esta métrica es aproximada ya que asume continuidad entre pódicos registrados.

7.2. Estrategias de Consolidación

7.2.1. 1. Agregación Global (Default)

Se aplican las fórmulas anteriores sobre **todos** los registros históricos de la patente P como un solo conjunto.

7.2.2. 2. Ponderación Mensual

Se calculan primero los promedios $X_{P,m}$ para cada mes m , y luego se ponderan por la cantidad de viajes en ese mes ($N_{P,m}$).

Sea X una variable cualquiera (ej. Velocidad, Conflict Rate). El valor final ponderado es:

$$X_{P,final} = \frac{\sum_m (X_{P,m} \cdot N_{P,m})}{\sum_m N_{P,m}} \quad (12)$$

Esto asegura que los meses con mayor actividad tengan mayor influencia, pero permite calcular métricas mensuales intermedias si se requiere análisis temporal.

8. Definición de Conductores Frecuentes

Para el entrenamiento del modelo de clustering, se separan los usuarios en:

- **Frecuentes:** Usuarios con suficiente historia para caracterizar su comportamiento. Criterios típicos:

- $N_P \geq 20$ pasadas totales.
 - Días activos ≥ 5 .
- **Infrecuentes:** Usuarios con pocos datos. No se usan para entrenar, pero pueden ser asignados a un cluster posteriormente usando el modelo entrenado (o quedar como "Desconocido").

9. Implementación en Python

La lógica de cálculo descrita anteriormente está implementada principalmente en el módulo `src/clustering.py`. A continuación se detalla el flujo de ejecución de las funciones clave para fines de auditoría.

9.1. Limpieza de Datos: `clean_flujos_for_clustering`

Esta función recibe el DataFrame de flujos brutos y realiza la preparación inicial:

1. **Validación de Columnas:** Se asegura que existan las columnas esenciales (timestamp, velocidad, pórtico, carril, patente).
2. **Limpieza de Patentes:** Se normalizan las patentes (alfanuméricos, mayúsculas) y se filtran aquellas con longitud inválida (fuera del rango $[5, 6]$).
3. **Filtrado Básico:** Se eliminan filas con datos nulos en campos críticos y se conservan solo los carriles 1, 2 y 3.
4. **Deduplicación:** Se eliminan duplicados exactos de la tupla (Patente, Pórtico, Carril, Timestamp). Si existen duplicados con diferentes velocidades, se prioriza determinísticamente (ordenamiento merge sort preservando el primero).
5. **Winsorization:** Se limitan los valores extremos de velocidad por grupo (Pórtico, Carril) usando los percentiles definidos (default 1% y 99%).

9.2. Cálculo de Features: Clusterization

Esta es la función principal que orquesta el cálculo de variables.

9.2.1. 1. Métricas Simples y Pre-cálculo

Se calculan métricas que no requieren ordenamiento temporal complejo:

- **Total de Pasadas (`total_passes`):** Conteo simple de registros por patente.
- **Velocidad Promedio (`avg_speed_kmh`):** Promedio aritmético de la velocidad.
- **Uso de Carril:** Se construye una tabla pivote para contar pasadas por carril y dividir por el total.

9.2.2. 2. Velocidad Relativa

- Se calcula la velocidad promedio del flujo en ventanas de 5 minutos por pórtico (`interval_speed_mean`).
- Se cruza este promedio con cada evento para calcular $\text{relative_speed} = v_i / \bar{V}_{5min}$.

9.2.3. 3. Métricas de Interacción (Headway, Conflicto)

Esta etapa es crítica y sensitiva al orden de los datos:

1. **Ordenamiento:** Los datos se ordenan estrictamente por Pórtico, Carril y Timestamp.
2. **Operaciones Vectorizadas (Shift):** Se crean columnas desplazadas (`shift(1)`) para comparar el evento actual con el vehículo inmediatamente anterior en el mismo carril/pórtico.
3. **Cálculo de Headway:** Diferencia de tiempo entre el evento actual y el anterior. Se descartan (NaN) los headways mayores a `max_headway_s` (60s).
4. **Cálculo de Conflicto:**
 - Se calcula $TTC = \frac{Headway \cdot v_{actual}}{v_{actual} - v_{anterior}}$.
 - Solo es válido si $v_{actual} > v_{anterior}$.
 - Se marca conflicto si $TTC < TTC_{max}$ (donde TTC_{max} depende del pórtico).
5. **Agregación:** Se suman y cuentan los eventos válidos por patente para obtener promedios.

9.2.4. 4. Cambios de Pista (Lane Changes)

- Se ordenan los datos por Patente y Timestamp.
- Se compara el carril actual con el anterior. Si son diferentes (y es la misma patente), se cuenta como cambio.
- La tasa se normaliza por $(N_P - 1)$.

9.2.5. 5. Ponderación (Si aplica)

Si se activa `monthly_weighting`, las métricas se calculan primero agrupando por Patente-Mes, y luego se hace un promedio ponderado por la cantidad de viajes del mes para obtener el valor final de la patente.

10. Alcance y metodología

Este informe documenta y audita el pipeline GNN implementado en el repositorio, con foco en:

- El flujo *tab a tab* de la aplicación Streamlit (`src/graph_builder_app.py`).
- La construcción del grafo, el modelo GNN, el entrenamiento y la evaluación (`src/graph.py`, `src/gat_model.py`, `src/gnn_main.py`, `src/train_pretrain.py`).
- Los mecanismos de balanceo: GraphSMOTE e ImGAGN (`src/graphsmote.py`, `src/imgagn.py`).
- La búsqueda de hiperparámetros (Optuna) y el control de métricas.

La auditoría incluye verificación de mejores prácticas, riesgos de fuga de información, uso de memoria/cálculo, y consistencia con las opciones de configuración (`src/config.py`).

11. Mapa general del pipeline

Resumen conceptual:

1. **Datos base:** pórticos, flujos y accidentes (`src/utils.py`).
2. **Feature engineering:** snapshot/flow (`src/snapshot_pipeline.py`, `src/features.py`).
3. **Feature selection:** selección manual/guiada (UI) y persistencia.
4. **Grafo:** nodos `pm`, aristas temporal/spatial/`st_fwd` y atributos (`src/graph.py`).
5. **Entrenamiento:** HeteroGAT + TemporalAggregator (opcional) (`src/gat_model.py`, `src/temporal_head.py`).
6. **Balanceo:** GraphSMOTE (embedding-space + $z \rightarrow x$ + edge-gen) y/o ImGAGN (adversarial) (`src/graphsmote.py`, `src/imgagn.py`).
7. **Evaluación:** métricas (F1, AUC, AUPRC, MCC), umbrales y calibración (`src/gnn_main.py`).
8. **Artefactos:** modelos, hparams JSON, grafos aumentados, historial (`Resultados/`, `gnn_history.jsonl`).

12. Pipeline *tab a tab* (Streamlit Graph Builder)

El flujo UI se implementa en `src/graph_builder_app.py` y organiza el trabajo en tabs.

12.1. Eventos

Función UI: `_render_events_tab`.

Objetivo: cargar y validar accidentes, normalizar columnas, limpiar y preparar el dataset de eventos.

- Carga de accidentes desde CSV/archivos seleccionados, persistencia en `st.session_state`.
- Uso de utilidades en `src/utils.py` para normalizar columnas y fechas.
- Soporte para generar reportes de *Puntos Negros* (posteriormente usados en *Graph*).

Implicación: este tab define el universo de accidentes, que afectará etiquetas futuras y cortes temporales.

12.2. Feature Engineering

Función UI: `_render_feature_engineering`.

Objetivo: generar o cargar features por nodo `pm`.

- Modos principales: *Snapshot Features* y *Flow 5 min* (ver `src/snapshot_pipeline.py`).
- Generación de features con `compute_pm_features` (`src/features.py`) y/o `SnapshotFeatureBuilder`.
- Salida persistida en DuckDB o CSV; sincronización de rango temporal y tramo.

Implicación: afecta dimensionalidad de nodos y distribución de atributos. La resolución temporal se gobierna con `DT_MINUTES` y parámetros en `src/config.py`.

12.3. Feature Explorer

Función UI: `render_feature_explorer`.

Objetivo: inspección visual, estadísticas y outliers de variables de entrada. **Implicación:** diagnóstico de calidad de features antes de construir el grafo.

12.4. Feature Selection

Función UI: `_render_feature_selection_tab`.

Objetivo: seleccionar subconjuntos de variables para entrenamiento, guardar listas y metadatos.

Implicación: cambia el espacio de entrada del GNN y la dimensionalidad de `pm.x`.

12.5. Graph

Funciones UI: `_render_create_graph`, `_render_load_graph`, `_render_in_memory_graph`.

Objetivo: construir o cargar el grafo heterogéneo de pórticos.

- Selección de tramo (manual o por puntos negros).
- Construcción de nodos `pm` (pórtico-minuto) con features normalizados.
- Generación de aristas temporales/espaciales y atributos de arista (ver Sección 13).
- Creación de `train/val/test_mask` con corte temporal (`_compute_time_cutoffs`).
- Guardado en `Resultados/highway_graph_*.pt` y en memoria (`st.session_state`).

12.6. Visual Graph

Función UI: `render_visual_graph_tab`.

Objetivo: visualizar topología, grados, y subgrafos para QA.

12.7. Network

Función UI: `_render_network_tab`.

Objetivo: configurar arquitectura y parámetros del modelo antes de entrenar. **Implicación:** fija opciones como `hidden_channels`, `num_heads`, `dropout`, `num_layers`, agregaciones `aggr1/aggr2` y `checkpointing`.

12.8. Optuna

Función UI: `_render_optuna_tab`.

Objetivo: búsqueda de hiperparámetros con Optuna (ver Sección 18). **Implicación:** almacena `optuna_hyperparams_*.csv` y gobierna la selección automática en entrenamiento.

12.9. Balance

Función UI: `_render_balance_tab`.

Objetivo: generar grafos balanceados con GraphSMOTE o ImGAGN.

- GraphSMOTE: requiere modelo entrenado para embeddings y decodificadores $\mathbf{z} \rightarrow \mathbf{x}$.
- ImGAGN: entrena generador y discriminador adversarial y crea nodos sintéticos.

Implicación: genera grafos con `is_synthetic`, actualiza `train_mask`, y puede cambiar el balance real.

12.10. Training

Función UI: `_render_training_tab`.

Objetivo: entrenar el GNN con opción de grafo balanceado, early stopping y evaluación automática.

Backend: `src/gnn_main.py::run_gat_training` y `src/train_pretrain.py::train_minibatch`.

12.11. Evaluación

Función UI: `_render_evaluation_tab`.

Objetivo: cargar modelos, evaluar métricas, aplicar umbral y calibración opcional. **Backend:**

`src/gnn_main.py::test` y utilidades de exportación.

12.12. History

Función UI: `_render_history_tab`.

Objetivo: registro de experimentos y trazabilidad (`Resultados/gnn_history.jsonl`).

12.13. Experiments

Función UI: `_render_gnn_experiments_tab`.

Objetivo: ejecutar lotes experimentales, exportar resultados, e importar ZIPs.

13. Modelo de datos del grafo

Nodos: `pm` (pórtico-minuto).

Targets: `pm.y` (etiqueta binaria), `pm.is_accident_pm` (accidente directo).

Máscaras: `train_mask/val_mask/test_mask/sequence_mask`.

13.1. Aristas y atributos

Aristas principales (`src/graph.py`):

- **temporal:** $P_{i,t} \rightarrow P_{i,t+1}$ (misma ubicación, siguiente tiempo).
- **spatial:** $P_{i,t} \rightarrow P_{i+1,t}$ (siguiente pórtico, mismo tiempo).
- **st_fwd** (opcional): $P_{i,t} \rightarrow P_{i+1,t+1}$.
- **spatial_back** (opcional): inversa de **spatial**.

Atributos de arista: diferencia de features entre destino y origen (Δx) y extras para **spatial** (gradientes, shock, distancia). En **temporal** y **st_fwd** se usan *extras* en cero para alinear dimensiones.

13.2. Etiquetado temporal

Cortes temporales: definidos por accidentes ordenados (`_compute_time_cutoffs`).

Máscaras: aplicadas por timestamps en minutos.

SequenceIndex: secuencias temporales con guard band y horizonte futuro (`src/snapshot_sequences.py`).

14. Arquitectura del modelo GNN

Modelo base: `src/gat_model.py::HeteroGAT`.

- **Capas:** `num_layers` bloques `HeteroConv` con `GATConvSaveAlpha`.
- **Agregación:** `aggr1` en la primera capa, `aggr2` en capas siguientes.
- **Atenciones:** capturadas cuando `XAI=1`, usadas para regularización $L2$.
- **Edge attributes:** se validan por batch; si faltan o desalinean, se rellenan con ceros.
- **Checkpointing:** `use_checkpointing` reduce memoria con recomputación.

14.1. Temporal head (opcional)

Clase: `src/temporal_head.py::TemporalAggregator`.

Idea: GRU sobre secuencias de embeddings recientes; mantiene cache de embeddings para nodos no presentes en el batch, evitando fuga de gradientes.

14.2. Registro de variantes GNN (encoder + temporal + aristas)

El pipeline ahora incorpora un **registro de variantes** controlado por `src/config.py` (`GNN_VARIANT`, `GNN_TEMPORAL_CACHE_STRATEGY`) y materializado en `src/gnn_main.py` mediante las funciones `_build_gnn_model`, `_build_temporal_head_for_variant` y `_prime_temporal_cache_if_needed`. Además, la UI en `src/graph_builder_app.py` expone una guía resumida en los tabs *Network*, *Training*, *Optuna* y *Evaluación*. La idea de diseño es separar:

- **Encoder espacial:** GAT heterogéneo base, GAT con codificador de aristas, o variante edge-aware.
- **Head temporal:** snapshot (sin secuencia), GRU, Transformer temporal o attention pooling.
- **Protocolo de cache:** incremental por batch o `global_epoch` (forward global por época para secuencias consistentes).

14.2.1. Idea general de comparabilidad

Todas las variantes comparten el mismo `SequenceIndex` (construido en `src/snapshot_sequences.py` a partir de `sequence_rows/target_rows`), el mismo grafo y los mismos splits temporales. Esto permite comparar arquitecturas sin cambiar la definición de etiqueta ni el horizonte de predicción.

Etiquetado “5 minutos antes” La lógica de etiquetado usa:

$$\text{intervalo positivo} = (t + \gamma, t + \gamma + H]$$

donde γ es `GUARD_BAND_MINUTES` y H es `HORIZON_MINUTES`. Para una configuración estricta de “5 minutos antes”, se recomienda:

- `GUARD_BAND_MINUTES` = 5
- `HORIZON_MINUTES` = 5 (o 10–20 para riesgo en ventana)

en `src/config.py`.

14.2.2. Variantes implementadas

Implementación: `src/gat_model.py`, `src/temporal_head.py`, `src/temporal_heads.py`, `src/gnn_main.py`.

Selección: mediante el string `GNN_VARIANT` (normalizado en `src/gnn_main.py::_normalize_gnn_variant`).

1. `gat_snapshot` (baseline): HeteroGAT sin head temporal. Clasifica con el embedding del snapshot actual.
2. `gat_gru`: HeteroGAT + `TemporalGRUHead` (`src/temporal_heads.py`).
3. `gat_transformer`: HeteroGAT + `TemporalTransformerHead` (self-attention con positional embedding aprendible).
4. `gat_attnpool`: HeteroGAT + `TemporalAttnPoolHead` (ablation ligera y barata computacionalmente).
5. `gat_edge_mlp`: `HeteroGATWithEdgeEncoder` (MLP por tipo de arista sobre `edge_attr`) sin head temporal.
6. `gat_edge_mlp_gru` y `gat_edge_mlp_transformer`: combinan codificador de aristas + head temporal.
7. `gnn_edge_aware`: `HeteroEdgeAware` (usa `TransformerConv`) sin head temporal.
8. `gnn_edge_aware_gru` y `gnn_edge_aware_transformer`: variante edge-aware con modelado temporal.

14.2.3. Codificación de aristas (`edge_attr`)

La variante `HeteroGATWithEdgeEncoder` agrega un MLP de aristas (`EdgeAttrEncoder`) antes de la atención. Motivación:

- los deltas Δx pueden tener escalas heterogéneas;
- un mapeo aprendido puede inducir una geometría más estable para la atención;
- permite estudiar de forma separada el efecto de “mejor `edge_attr`” vs. “mejor head temporal”.

14.2.4. Heads temporales y ensamblado de secuencias

Los heads temporales reutilizan la infraestructura de `TemporalAggregator`:

- `update_cache(...)` para cache global o parcial de embeddings.
- `_build_sequence_batch(...)` para reconstruir tensores $[B, L, D]$ usando `sequence_rows`.

Esto permite cambiar el operador temporal (GRU/Transformer/Attention Pooling) sin cambiar el contrato del entrenamiento en `src/train_pretrain.py` ni la firma de `forward`.

14.2.5. Estrategias de cache temporal (punto crítico)

- `incremental`: cache por batch (histórico, menor costo, más dependiente del sampler).
- `global_epoch`: priming por época con forward global (`compute_epoch_embeddings`), luego training/eval usan cache consistente.

Recomendación para artículo/ciencia: usar `global_epoch` para que las secuencias temporales tengan la misma referencia de embeddings dentro de una época.

14.2.6. Matriz experimental recomendada

1. `gat_snapshot` (baseline espacial)
2. `gat_gru`
3. `gat_transformer`
4. `gat_edge_mlp_gru`
5. `gnn_edge_aware_gru` (o `gnn_edge_aware_transformer`)

Mantener semilla, split temporal, `SequenceIndex` y protocolo de evaluación constantes.

14.2.7. Métricas para eventos raros

Para comparación científica y operativa, priorizar:

- **PR-AUC / AUPRC**: métrica principal en desbalance severo.
- **ROC-AUC**: referencia secundaria.
- **Recall@FAR/FPR objetivo**: útil para restricciones operacionales.
- **MCC**: robusta cuando hay fuerte asimetría de clases.

14.2.8. Trazabilidad de variantes

El entrenamiento y HPO guardan `gnn_variant` y `variant_tag` en sidecars de hiperparámetros, checkpoints y logs estructurados (`Resultados/gat_model_BEST*.pt`, `_hparams.json`, Optuna CSVs). Esto evita mezclar checkpoints de arquitecturas incompatibles.

15. Entrenamiento y optimización

Punto de entrada: `src/gnn_main.py::run_gat_training`.

15.1. Flujo del entrenamiento

1. **Dispositivo**: `get_auto_device()` (MPS > CUDA > CPU).
2. **Carga de datos**: `loaded_obj['data']`.
3. **ImGAGN automático** (si `AUTO_IMGAGN_PRETRAIN=1`).
4. **GraphSMOTE**: elección por usuario/flag y detección de sintéticos existentes.
5. **HPO**: selección de `optuna_hyperparams_*.csv` o búsqueda nueva.
6. **Modelo**: fábrica `_build_gnn_model` selecciona encoder/temporal head según `gnn_variant` y adjunta el head temporal cuando aplica.
7. **Decodificadores $z \rightarrow x$** : entrenados si GraphSMOTE activo (`train_z2x_decoders`).
8. **Edge generator**: `RelEdgeGen` para reconstrucción y aristas sintéticas.

9. **Pérdida:** CrossEntropy o FocalLoss (con/ sin pesos).
10. **Scheduler:** OneCycleLR.
11. **Loop:** entrenamiento por mini-batches, validación, early stopping, guardado de mejor modelo.

15.2. Train-minibatch y regularización

Función: `src/train_pretrain.py::train_minibatch`.

- Clasificación sobre pm (primeros `batch_size` del `NeighborLoader`).
- Pérdida total:

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_{edge} \mathcal{L}_{edge} + \lambda_{att} \mathcal{L}_{att}.$$

- $\mathcal{L}_{edge}$: reconstrucción de aristas con muestreo negativo (si `edge_gen`).
- $\mathcal{L}_{att}$: L2 sobre atenciones cuando `XAI=1`.
- Gradiente acumulado (`accumulation_steps=2`) y `grad_clip`.
- Soporte AMP (solo CUDA).

16. GraphSMOTE: síntesis de nodos y aristas

Módulo: `src/graphsmote.py`.

Principio: generar nodos sintéticos en el espacio embedding y decodificarlos a features reales.

16.1. Paso 1: embeddings

- `get_embeddings_minibatch` (para balance offline en UI) o `compute_train_embeddings` (en entrenamiento, sin fuga).
- Normalización ℓ_2 por nodo.

16.2. Paso 2: SMOTE en embeddings

Se aplica SMOTE en el conjunto minoritario:

$$\mathbf{z}_{syn} = (1 - \alpha) \mathbf{z}_i + \alpha \mathbf{z}_j, \quad \alpha \sim U(0, 1).$$

- k-NN sobre embeddings minoritarios (CPU cache o GPU si cabe).
- Parámetros clave: `GRAPHSMOTE_K`, `smote_k`, `random_state`.

16.3. Paso 3: decodificadores $\mathbf{z} \rightarrow \mathbf{x}$

Clase: `Z2XDecoders`.

Entrenamiento: `train_z2x_decoders` con SmoothL1/MSE, early stopping y validación.

- Un MLP por tipo de nodo: $\mathbf{x}_{syn} = f_{ntype}(\mathbf{z}_{syn})$.
- Se respetan `train/val_mask` y se evita usar nodos fuera del split cuando está disponible.

16.4. Paso 4: inserción de nodos

- Concatenar `x` y `y` sintéticos a `pm.x` y `pm.y`.
- Actualizar `train/val/test_mask` (por defecto nuevos nodos van a `train`).
- Crear/actualizar `is_synthetic` y `is_accident_pm`.

16.5. Paso 5: generación de aristas sintéticas

Generador: `RelEdgeGen` (parámetro por relación).

Candidatos reales:

- `GRAPHSMOTE_CONNECT=1`: top-K en `train_mask`.
- `GRAPHSMOTE_CONNECT=0`: vecinos a 1 salto de accidentes reales.

Dirección:

- `BIDIRECTION=1`: real $\leftrightarrow$ sintético.
- `BIDIRECTION=0`: solo real $\rightarrow$ sintético.

Atributos de arista: se computan como Δx entre nodos destino y origen (con `delta_feature_idx` si existe) y se alinean a la dimensionalidad existente.

16.6. Modos de operación

- **Offline:** `augment_graph_offline_once` aumenta una vez y usa el grafo para entrenar.
- **Online:** `refresh_synthetics_online` regenera nodos cada `SMOTE_EVERY_N_EPOCHS`.
- **None:** sin aumentación.

17. ImGAGN: generación adversarial

Módulo: `src/imgagn.py`.

Idea central: generar nodos sintéticos como combinaciones convexas de nodos minoritarios *de entrenamiento* y entrenar un discriminador adversarial.

17.1. Paso 1: homogenización

`to_homogeneous_if_needed` transforma `HeteroData` a grafo homogéneo (target `pm`), preservando `edge_attr` y `delta_feature_idx` si existen.

17.2. Paso 2: definir cantidad de sintéticos

$$\lambda_1 = \frac{n_{min} + n_g}{n_{maj}}$$
$$\Rightarrow n_g = \text{máx}(0, \lambda_1 n_{maj} - n_{min})$$

Se limita con `max_new_nodes` para evitar OOM.

17.3. Paso 3: generador

Clase: `ImGAGNGenerator`.

- Entrada: ruido $\mathbf{z} \sim \mathcal{N}(0, I)$.
- Salida: logits por nodo minoritario de entrenamiento.
- Pesos $T = \text{softmax}(G(z))$ y features sintéticos $X_g = TX_{min}$.
- Aristas: top-K conexiones a nodos minoritarios reales según T .

17.4. Paso 4: discriminador

Clase: `ImGAGNDiscriminator` (GCN + 2 cabezas).

- **Head real/fake** (BCE).
- **Head minority/majority** (BCE).
- Término de separación `_mm_separation` para alejar medias minoritaria y mayoritaria.

17.5. Paso 5: entrenamiento adversarial

Por cada `epoch`:

1. Muestrear ruido y construir sintéticos (sin gradientes).
2. Construir grafo aumentado y entrenar D por `d_steps` minibatches.
3. Recalcular X_g con gradientes y optimizar G para:

$$\mathcal{L}_G = \mathcal{L}_{rf} + \mathcal{L}_{mi} + \mathcal{L}_{di},$$

donde $\mathcal{L}_{rf}$ empuja a *real*, $\mathcal{L}_{mi}$ empuja a *minority*, y $\mathcal{L}_{di}$ acerca embeddings a centroides minoritarios.

Escalabilidad: usa `NeighborLoader` si está disponible; si no, full-batch.

17.6. Paso 6: grafo final

Se reconstruye el grafo aumentado con el generador final y se devuelven: `x_aug`, `edge_index_aug`, `edge_attr_aug`, y la porción de nodos generados.

18. Optuna (búsqueda de hiperparámetros)

Función: `src/gnn_main.py::objective`.

- Objetivo: maximizar F0.5 en validación (prioriza precisión).
- Espacio: `hidden_channels`, `num_heads`, `dropout`, `num_layers`, `aggr1/aggr2`.
- Optimizadores: Adam/AdamW/RAdam (configurable).

- Con GraphSMOTE: busca `smote_k`, `target_pos_ratio`, `smote_every_n_epochs`, `lambda_edge`.
- Sin GraphSMOTE: FocalLoss con `focal_alpha`, `focal_gamma`.

Muestreo de seeds (train/val):

- *Resamplear seeds por epoch*: cambia el subset en cada epoch (mide robustez).
- *Fijar subset por trial*: mantiene el mismo subset durante el trial (reduce varianza).
- *Cargar todo el grafo en memoria (sin muestreo)*: usa todos los nodos `train/val` (equivale a `sample_train_nodes=0` y `sample_val_nodes=0`); mayor uso de memoria.

19. Evaluación, calibración y artefactos

- **Evaluación**: `test` calcula F1, AUC, AUPRC, MCC, matriz de confusión.
- **Umbral**: `pick_tau_fbeta` busca umbral τ en validación.
- **Calibración**: Platt scaling opcional (`AUTOCALIBRATE_PROBS`).
- **Guardado**: modelos `gat_model_BEST*.pt`, `GraphSMOTE_embeddings_model*.pt`, `sidecar_hparams.json`.
- **Hash del grafo**: `_calculate_graph_hash` para trazabilidad.

20. Auditoría técnica y mejores prácticas

20.1. Fortalezas observadas

- Separación temporal para `train/val/test` y uso de `sequence_mask` reduce fuga temporal.
- GraphSMOTE en entrenamiento usa embeddings de `train` (`compute_train_embeddings`).
- `NeighborLoader` limita memoria y permite entrenamiento en grafos grandes.
- Mecanismos de limpieza/*coalesce* para evitar inconsistencias en `edge_attr`.
- Guardado de `hparams` y `run_id` para reproducibilidad.

20.2. Riesgos o puntos críticos (actualizado)

- **Fuga en balanceo manual (mitigado)**: `run_graphsmote` ahora restringe embeddings/pool minoritario a `train_mask`. Mantener verificación en auditorías.
- **Dependencia de BIDIRECTION**: dirección de aristas sintéticas afecta mensajes y puede sesgar la difusión de señales. Se mantiene sin cambios por ahora.
- **Acumulación de gradientes (ajustable)**: `accumulation_steps` ahora es configurable (config/UI). Aún requiere tuning según memoria/hardware.
- **Balanceo + pesos de clase**: se deshabilita pesos con GraphSMOTE/ImGAGN (correcto), pero requiere monitoreo de métricas en validación.

20.3. Recomendaciones priorizadas (estado actual)

1. **Implementado:** en `run_graphsmote` (balance manual) se limita a `train_mask` para evitar fuga potencial.
2. **Implementado:** exponer `accumulation_steps` en `config/UI`; ajustar según memoria disponible.
3. **Pendiente/decisión:** no modificar `BIDIRECTION` por ahora; documentar el racional y evaluar impacto en AUPRC/F1 si se cambia en el futuro.
4. **Implementado:** en `ImGAGN` registrar `best_train_recall` junto a `seed` y `config` para trazabilidad.

21. Anexo: referencias a funciones y archivos

- UI principal: `src/graph_builder_app.py`.
- Construcción del grafo: `src/graph.py`.
- Modelo GNN: `src/gat_model.py`.
- Temporal head: `src/temporal_head.py`.
- Entrenamiento/evaluación: `src/gnn_main.py`, `src/train_pretrain.py`.
- GraphSMOTE: `src/graphsmote.py`.
- ImGAGN: `src/imgagn.py`.
- Configuración: `src/config.py`.

22. Implementación de MA-AIRL

22.1. Objetivo y alcance

Se implementó una versión funcional del algoritmo Multi-Agent Adversarial Inverse Reinforcement Learning (MA-AIRL) siguiendo el marco descrito en `src/MAIRL.pdf`. El objetivo es aprender, por cada clúster de conductores, una política estocástica y una función de recompensa que expliquen las demostraciones observadas en los datos agregados de clústeres, y luego exportar parámetros de *vTypes* para SUMO.

22.2. Arquitectura general

La implementación se dividió en dos capas:

- **Núcleo MA-AIRL** en `src/marl_core.py`: datos expertos, normalización, redes (política, recompensa y potencial), entrenamiento y exportación de parámetros.
- **Interfaz** en `src/multi_agent_rl_app.py`: carga de datos, selección de *features*, control de hiperparámetros, ejecución del entrenamiento y visualización de métricas.

22.3. Datos expertos y construcción del estado

Los datos de entrada son archivos CSV de clústeres con la columna `cluster_label`. Cada fila se interpreta como una muestra experta, con un estado s compuesto por columnas numéricas seleccionadas (por defecto: `avg_speed_kmh`, `avg_headway_s`, `lane_change_rate`). Se implementó:

- **Selección de features** desde la UI para definir el vector de estado.
- **Estandarización** con un `StandardScaler` propio (media y desviación por columna).
- **Separación por agente** agrupando por `cluster_label`.

22.4. Acciones expertas (proxy de SUMO)

MA-AIRL requiere pares (s, a) . Como el CSV no contiene acciones reales, se construyó un **proxy de acción** para cada muestra:

- Se mapean métricas a parámetros de car-following: `maxSpeed`, `accel`, `decel`, `minGap`, `sigma`, `impatience`.
- Se aplican transformaciones heurísticas y se recortan a los límites de `PARAM_BOUNDS`.
- Las acciones se normalizan a $[-1, 1]$ para entrenamiento estable.

Esto permite entrenar MA-AIRL aunque no existan trayectorias con acciones explícitas.

22.5. Modelos de MA-AIRL

Para cada agente (clúster) se instancian tres redes:

- **Política gaussiana** $q_\theta(a | s)$: MLP que produce μ y $\log \sigma$.
- **Estimador de recompensa** $g_\omega(s, a)$: MLP sobre el par estado-acción.
- **Potencial** $h_\phi(s)$: MLP sobre el estado.

Se implementó la forma estructurada:

$$f_{\omega, \phi}(s, a, s') = g_\omega(s, a) + \gamma h_\phi(s') - h_\phi(s)$$

22.6. Función discriminadora y objetivo

Siguiendo la Ecuación (11) del paper, el discriminador se define como:

$$D_\omega(s, a) = \frac{\exp(f_\omega(s, a))}{\exp(f_\omega(s, a)) + q_\theta(a | s)}$$

El entrenamiento usa:

- **Paso del discriminador**: maximiza $\mathbb{E}_{X_E}[\log D] + \mathbb{E}_{X_\pi}[\log(1 - D)]$.
- **Paso del generador (política)**: maximiza $\mathbb{E}_{X_\pi}[f_\omega(s, a) - \log q_\theta(a | s)]$ (Ecuación 12).

22.7. Proxy temporal

Los datos no incluyen transiciones temporales explícitas (s, s') . Para mantener la estructura MA-AIRL se aplicó un **proxy de un paso**:

$$s' = s$$

De esta forma se conserva la forma del potencial sin requerir secuencias completas. Esta decisión se documentó en el código como aproximación.

22.8. Métricas de entrenamiento

Durante el entrenamiento se reportan:

- **disc_loss**: pérdida del discriminador.
- **policy_loss**: pérdida de la política.
- **reward_mean**: promedio de f_w en muestras expertas.

Se guardan en memoria para graficado y análisis posterior.

22.9. Exportación de parámetros a SUMO

Al finalizar, se obtienen parámetros por agente evaluando la política en el estado medio y des-normalizando a rangos SUMO. Se exporta un archivo:

- `mairl_vtypes.add.xml` con un `vType` por clúster.

Esto permite inyectar los parámetros aprendidos en simulaciones SUMO reales.

22.10. Integración en la UI

Se añadió un selector de algoritmo:

- **MA-AIRL (paper)**: entrenamiento adversarial con datos expertos.
- **CEM (legacy)**: método anterior basado en cross-entropy con SUMO.

También se agregó control de hiperparámetros (batch size, γ , *learning rates*, tamaño de red y dispositivo).

22.11. Archivos modificados

- `src/marl_core.py`: núcleo MA-AIRL (datos, redes, entrenamiento, exportación).
- `src/multi_agent_rl_app.py`: UI y flujo de entrenamiento MA-AIRL.

22.12. Limitaciones actuales y próximos pasos

- La implementación usa $s' = s$ por falta de secuencias temporales explícitas.
- Las acciones expertas son heurísticas derivadas de métricas agregadas.
- El siguiente paso recomendado es construir trayectorias reales desde SUMO o datos temporales para estimar s' y acciones reales.